A Lightweight Lexical-Metadata Hybrid for Duplicate Bug Report Retrieval

Shah Nadim Kamran Rian
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh
Email: shah.rian.252@northsouth.edu

***Abstract*—**Duplicate bug reports are a recurring problem in large software repositories because they increase triage effort, split technical discussion across multiple reports, and delay issue resolution. Recent work often focuses on more complex semantic models, but simpler retrieval systems are still useful when interpretability, cost, and reproducibility matter. In this study, I evaluate a hybrid model for duplicate bug report retrieval that combines TF–IDF lexical similarity with a component-level metadata signal through weighted score fusion.

The evaluation uses a processed 10,000-report subset from the MSR 2013 Eclipse and Mozilla defect tracking dataset. I treat this as a controlled subset experiment rather than a full-corpus benchmark. Four configurations are compared under the same proxy relevance protocol: TF–IDF, latent semantic analysis (LSA), metadata-only matching, and the proposed lexical-metadata hybrid. The results show that LSA achieves the strongest top-rank performance, with the highest MRR and Hit@1, while the proposed hybrid obtains the best Recall@5 and Recall@10. These results suggest that the hybrid model is most useful when the goal is to show a better short list of possible duplicate reports, although it should not be interpreted as universally superior across all ranking metrics.

***Index Terms*—duplicate bug report retrieval, bug triage, software maintenance, TF–IDF, metadata similarity, lexical-metadata hybrid**

I. Introduction

Software maintenance relies heavily on issue trackers such as Bugzilla, JIRA, and GitHub Issues. As repositories grow, they often accumulate duplicate bug reports that describe the same underlying problem using different words, levels of detail, or user perspectives. These duplicates increase the effort required for manual triage, spread useful technical discussion across multiple reports, and slow down issue resolution. Duplicate bug handling is therefore both a maintenance problem and a retrieval problem over short, noisy, and technical text.

The difficulty comes from the nature of bug report data. Two reports may describe the same failure with different terminology, while two non-duplicate reports may still share many technical words because they involve related components, products, or symptoms. As a result, simple keyword matching is often not enough, but highly complex models may also be unnecessary or impractical in resource-limited settings.

Duplicate bug report retrieval has been studied using information retrieval, supervised learning, and more recent neural representation methods. Strong semantic models can improve matching quality in many settings, but they also introduce higher implementation complexity, memory requirements, and inference cost. A lightweight and transparent baseline is still valuable when the goal is to build a reproducible system that can be understood, tested, and deployed with limited resources.

This paper studies a compact hybrid design using two signals that are usually available in bug tracking data. The first signal is TF–IDF lexical similarity, which provides a strong sparse-text baseline. The second signal is a simple metadata cue based on component agreement. The goal is not to claim that this design replaces semantic retrieval. Instead, I ask a narrower question: can a small amount of metadata improve short-list retrieval coverage while keeping the system simple and interpretable?

The contributions of the paper are fourfold:

- A duplicate-bug retrieval approach is presented by combining TF–IDF lexical similarity with a component-level metadata signal.
- A controlled comparison is conducted among TF–IDF, LSA, metadata-only ranking, and lexical-metadata hybrid fusion on the same processed 10,000-report subset.
- The results show a clear trade-off: LSA achieves the strongest MRR and Hit@1, while the proposed hybrid achieves the best Recall@5 and Recall@10.
- The paper provides a low-cost and reproducible baseline for settings where computational simplicity and interpretability are important, while also stating the limits of the proxy relevance evaluation.

II. Related Work

Duplicate bug report detection has been investigated for more than a decade from several directions. Early work showed that duplicate detection can be formulated as a retrieval problem over bug reports. Runeson *et al.* used natural language processing to detect duplicate defect reports and showed that textual similarity can be effective in practice [1]. Wang *et al.* extended this direction by combining natural language and execution information, highlighting the value of complementary signals [2].

A. Information Retrieval Methods

Classical information retrieval remains an important foundation for duplicate bug report retrieval. TF–IDF is widely used because it is simple, interpretable, and effective in domains where informative technical terms are unevenly distributed [9],

[10]. BM25 and related probabilistic ranking models are also well-established in document retrieval [11]. These methods are attractive for bug report collections because they can be indexed efficiently, queried quickly, and explained without relying on opaque model internals.

The main limitation of lexical ranking is its dependence on token overlap. This becomes challenging when different users describe the same bug with different vocabulary. Issue trackers often contain interface terms, version identifiers, partial stack traces, informal wording, and product-specific language. Therefore, lexical models may work well for near-duplicates but struggle when semantic similarity is not accompanied by strong word overlap.

B. Traditional Supervised Approaches

Other studies have treated duplicate detection as a predictive task. Jalbert and Weimer showed that supervised learning over bug tracker data can support automated duplicate identification [3]. Sun *et al.* later introduced discriminative models designed specifically for ranking duplicate bug reports [4], [5]. These studies were influential because they addressed duplicate detection as ranked retrieval instead of only binary classification.

A benefit of supervised approaches is their ability to combine heterogeneous evidence such as title similarity, description overlap, component matches, severity agreement, and report history. Their drawback is that they often depend on manually designed features and repository-specific engineering. When different projects use different tracker schemas, such feature engineering can become difficult to maintain.

C. Latent Semantic and Representation-Based Methods

LSA was introduced to reduce sparsity by projecting documents into a lower-dimensional semantic space [12]. In some text domains, this smoothing can reveal useful latent structure. In bug reports, however, the benefit is not guaranteed because the text is often short, technical, and irregular. Dimensionality reduction can sometimes remove details that are important for distinguishing one bug from another. For this reason, LSA is evaluated as a baseline in the present study rather than assumed to be better or worse in advance.

D. Repository Mining and Metadata-Aware Studies

Bug repository mining research has shown that issue trackers contain useful structure beyond free text. The Eclipse and Mozilla defect tracking dataset introduced by Lamkanfi, Pérez, and Demeyer remains a public resource for mining bug information [6]. Prior studies have also shown that bug report structure and metadata can describe maintenance behavior and developer information needs [7], [8]. These findings are relevant because duplicate bug retrieval is rarely a pure text problem. Structured fields such as product, component, platform, severity, and operating environment may provide useful context even when text alone is ambiguous.

E. Neural and Transformer-Based Approaches

More recent work has increasingly used contextual encoders. BERT [13] and Sentence-BERT [14] demonstrate strong semantic modeling ability for many matching tasks, including tasks related to software engineering. These methods are valuable reference points, but they usually require larger memory budgets, heavier inference pipelines, and more engineering effort than classical sparse retrieval. The present study therefore focuses on a low-cost and interpretable baseline, while treating stronger neural semantic retrieval as an important future comparison.

F. Positioning of the Present Study

This paper is intentionally designed as a lightweight study. It does not claim to outperform full transformer-based retrieval systems. Instead, it investigates whether a small lexical-metadata fusion can improve candidate coverage without requiring expensive semantic infrastructure. This positioning keeps the study practical: the goal is to provide a transparent baseline that can be reproduced and understood without a heavy retrieval pipeline.

III. Problem Formulation

Let the bug report collection be

$$\mathcal{B} = \{b_1, b_2, \ldots, b_n\}.$$

Each bug report contains a processed text field and selected metadata:

$$b_i = (T_i, M_i),$$

where T_i denotes the textual representation and M_i includes structured attributes such as product and component.

Given a query report b_q, the task is to rank candidate reports

$$\mathcal{R}_q = \langle b_{j_1}, b_{j_2}, \ldots, b_{j_k} \rangle$$

so that relevant reports appear as early as possible.

A. TF–IDF Similarity

Each report text is converted into a TF–IDF vector:

$$v_i = \mathrm{TFIDF}(T_i).$$

The lexical similarity between two reports is cosine similarity:

$$S_{\mathrm{tfidf}}(i,j) = \frac{v_i \cdot v_j}{\|v_i\| \, \|v_j\|}. \tag{1}$$

B. LSA Similarity

For the LSA baseline, the TF–IDF matrix is projected into a lower-dimensional latent space using truncated SVD. Let X be the TF–IDF matrix and let X_r denote its rank-r approximation. The LSA similarity is computed with cosine similarity:

$$S_{\mathrm{lsa}}(i,j) = \cos(x_i^{(r)}, x_j^{(r)}), \tag{2}$$

where $x_i^{(r)}$ denotes the latent representation of report i after rank-r truncated SVD.

C. Metadata Similarity

Metadata is used conservatively in the final hybrid model. The metadata score is based on component agreement:

$$S_{\text{meta}}(i,j) = \begin{cases} 1, & \text{if component}_i = \text{component}_j, \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

D. Normalization and Hybrid Fusion

Because lexical and metadata scores may operate on different scales, both similarity matrices are normalized to the range $[0, 1]$:

$$S' = \frac{S - \min(S)}{\max(S) - \min(S)}. \tag{4}$$

In the present implementation, this normalization is applied globally over each similarity matrix before fusion rather than separately for each query row.

The final hybrid similarity is a weighted sum of normalized scores:

$$S_{\text{hybrid}}(i,j) = \alpha S'_{\text{tfidf}}(i,j) + \beta S'_{\text{meta}}(i,j), \tag{5}$$

subject to

$$\alpha + \beta = 1, \qquad \alpha, \beta \geq 0. \tag{6}$$

In the final reported configuration,

$$\alpha = 0.85, \qquad \beta = 0.15. \tag{7}$$

These weights were selected through small-scale exploratory tuning. I kept most of the weight on TF–IDF because the lexical signal was the strongest base signal, while the metadata score was added only as a modest adjustment for candidate coverage.

E. Evaluation Metrics

Ranking quality is measured using Mean Reciprocal Rank (MRR), Hit@1, Recall@5, and Recall@10. Let $\text{rank}(q)$ denote the rank position of the first relevant candidate for query q. Then

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}(q)}, \tag{8}$$

$$\text{Hit@1} = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{I}(\text{rank}(q) = 1), \tag{9}$$

and for a cutoff k,

$$\text{Recall@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{|Top_k(q) \cap Rel(q)|}{|Rel(q)|}, \tag{10}$$

where $Rel(q)$ denotes the relevant set for query q.

IV. Methodology

A. Design Rationale

The proposed system follows the idea that duplicate bug retrieval can benefit from more than one simple signal. Lexical similarity is fast and often strong when reports share vocabulary. Metadata is weaker as a complete retrieval method, but it can still provide useful context when reports belong to related parts of the repository. This study therefore evaluates whether a simple hybrid can improve short-list retrieval coverage while keeping the method understandable and inexpensive to run.

B. Data Preparation

The source data is derived from the MSR 2013 Eclipse and Mozilla defect tracking dataset [6]. The public dataset is available online at https://zenodo.org/records/268443. For this experiment, I used a processed subset of 10,000 reports to support repeated local runs on available laptop hardware. The study is therefore scoped as a controlled subset experiment rather than a full-corpus benchmark.

The final working fields were `text`, `product`, and `component`. The `text` field was formed from the processed short textual description available in the working dataset. Preprocessing was intentionally light:

- lowercasing,
- removal of empty entries,
- removal of very short or low-information records.

The subset was created after preprocessing and resource-constrained filtering. I limited it to 10,000 reports because computing full pairwise similarity on a larger subset became slow on my local machine. This size made the experiment easier to repeat while still being large enough to compare the four retrieval settings. Therefore, the results should be interpreted as controlled subset outcomes rather than full-dataset claims.

C. Feature Extraction and Model Variants

Four configurations were evaluated on the same processed subset.

TF–IDF baseline: cosine ranking over sparse TF–IDF vectors. The implementation limits the vectorizer to 5000 features so that the representation remains compact enough for repeated local testing.

LSA baseline: cosine ranking after truncated SVD over the textual matrix. The reduced latent space uses 16 components, which is consistent with the study's lightweight design goal.

Metadata-only model: ranking by component agreement. This simple model is included to show how far structured metadata can go without lexical evidence.

Hybrid model: weighted fusion of TF–IDF and metadata similarity using Eq. (5).

D. Leakage Prevention and Relevance Proxy

A key evaluation design choice was to avoid direct feature leakage. Relevance groups were built at the product level rather than by product and component together. In implementation terms, the grouping function is:

```
groups = df["product"]
```

while metadata similarity uses component equality only. This prevents exactly the same feature from defining both the relevance group and the metadata ranking score.

At the same time, this design should be interpreted carefully. Product-level grouping is only a proxy for duplicate relevance, not true duplicate-pair ground truth. Reports from the same product are not necessarily duplicates. For this experiment, the grouping is useful for controlled comparison, but it is not a replacement for evaluation on verified duplicate pairs.

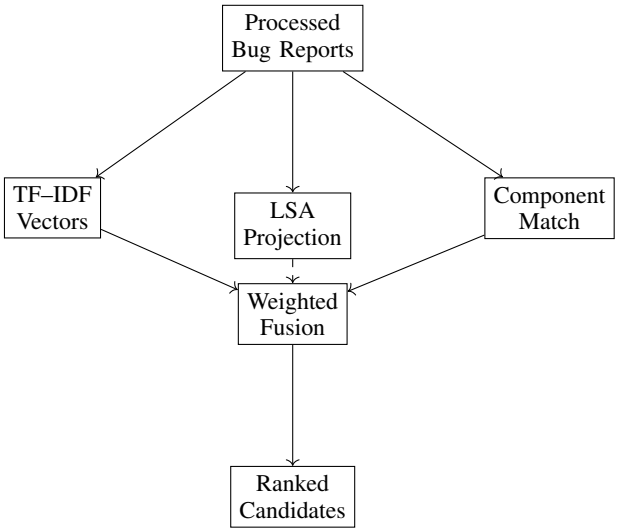

Fig. 1. Overall retrieval pipeline. The final hybrid reported in the paper fuses TF–IDF and metadata; LSA is evaluated as a separate baseline rather than included in the final fusion.

E. Pipeline Overview

After cleaning bug reports, the pipeline computes three signals. TF–IDF and LSA operate on textual representations, while metadata matching uses component-level contextual information. The final hybrid uses TF–IDF and metadata only. LSA is kept as a separate baseline because the purpose of the hybrid is to test a direct lexical-metadata fusion rather than to combine all available signals.

V. Experimental Setup

A. Dataset Scope

The Eclipse and Mozilla defect tracking dataset [6] is the source data. Instead of using the entire collection, the reported experiments use a processed 10,000-report subset. This decision supports a controlled lightweight study and avoids making unsupported full-corpus claims.

B. Evaluation Protocol

The evaluation reports MRR, Hit@1, Recall@5, and Recall@10. MRR and Hit@1 reflect top-rank quality, while Recall@5 and Recall@10 indicate whether relevant reports remain visible within a short candidate list. This is useful for duplicate triage because maintainers may inspect several suggestions rather than only the first result.

C. System Configuration

Table I summarizes the working experimental configuration used in the current study.

D. Future Strengthening Steps

Two natural extensions remain outside the numerical scope of this version:

- **Temporal evaluation**, where candidate duplicates are restricted to earlier reports only.
- **Statistical significance testing**, using query-level paired comparisons.

These are discussed as future strengthening steps rather than presented as unverified numerical claims.

TABLE I
Experimental setup summary.

Item	Setting
Source dataset	MSR 2013 Eclipse + Mozilla
Working subset size	10,000 reports
Subset rationale	hardware-constrained repeated local runs
Text model	TF–IDF
TF–IDF vocabulary cap	5000 features
Latent baseline	Truncated SVD (LSA)
LSA dimension	16 components
Metadata field	Component
Grouping field	Product
Hybrid weights	0.85 TF–IDF, 0.15 metadata
Normalization	global matrix normalization
Metrics	MRR, Hit@1, Recall@5, Recall@10

TABLE II
Performance comparison of the evaluated models on the 10,000-report subset.

Model	MRR	Hit@1	Recall@5	Recall@10
TF–IDF	0.599683	0.487845	0.001311	0.002709
LSA	**0.652232**	**0.508379**	0.001425	0.002881
Metadata	0.454691	0.201086	0.001167	0.002751
Hybrid	0.625442	0.506490	**0.001512**	**0.003229**

VI. Results and Discussion

A. Main Results

Table II reports the main comparison on the processed 10,000-report subset.

B. Precision-Oriented Comparison

Figure 2 summarizes the top-rank behavior of the four evaluated models. On this 10,000-report subset, LSA achieves the strongest MRR and Hit@1. TF–IDF remains a strong lexical baseline, and the hybrid model stays close to LSA in Hit@1, but it does not produce the highest top-rank score. This distinction matters because MRR and Hit@1 emphasize whether the first relevant candidate appears very early in the ranking.

C. Coverage-Oriented Comparison

Figure 3 highlights the short-list retrieval behavior. The hybrid model achieves the best Recall@5 and Recall@10 among the evaluated configurations. This suggests that adding a small metadata signal helps more with candidate coverage than with first-rank dominance. In practical triage, this can still be useful because a maintainer may inspect a short list of ranked reports rather than only the first suggestion.

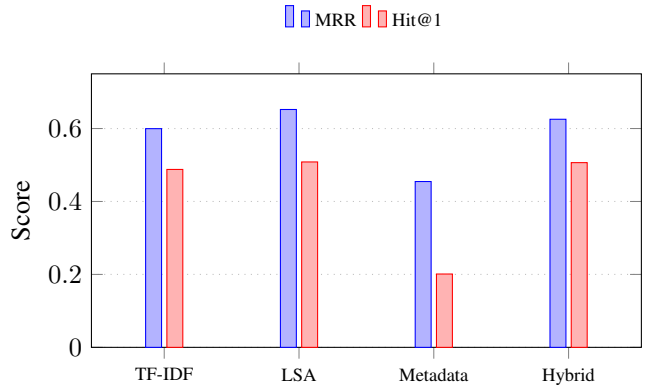

Fig. 2. Top-rank retrieval quality on the 10,000-report subset. LSA achieves the strongest MRR and Hit@1, while the hybrid remains competitive.

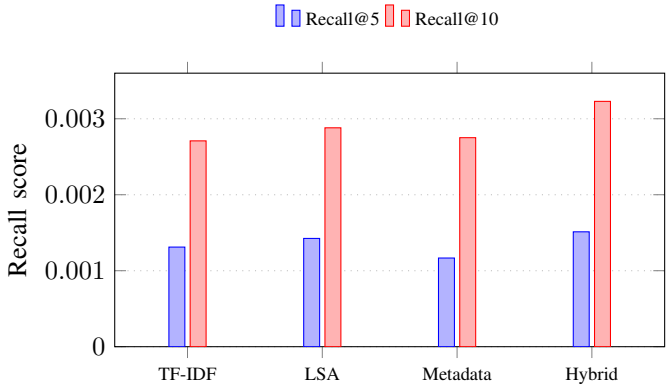

Fig. 3. Short-list retrieval coverage on the 10,000-report subset. The hybrid achieves the strongest Recall@5 and Recall@10 among the evaluated models.

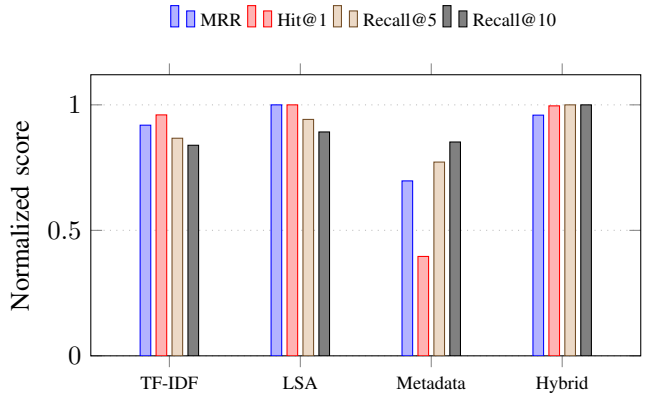

Fig. 4. Normalized metric profile. Each metric is divided by its best observed value, showing that LSA leads top-rank quality while the hybrid leads short-list recall.

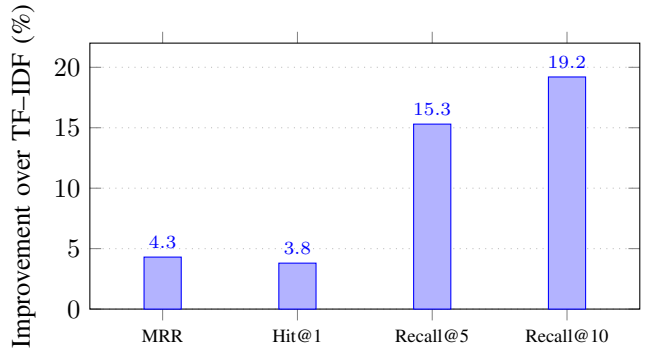

Fig. 5. Relative improvement of the hybrid model over the TF–IDF baseline. The largest gains occur in the recall-oriented metrics.

D. Normalized Metric Profile and Hybrid Gain

Because the four metrics operate on different numeric ranges, Figure 4 reports a normalized metric profile. Each score is divided by the best score achieved for that metric, so a value of 1.0 indicates the strongest model for that metric. This view makes the trade-off clearer: LSA leads the top-rank metrics, while the hybrid leads both recall-based metrics.

Figure 5 compares the hybrid directly against the TF–IDF baseline. The hybrid improves all four reported metrics relative to TF–IDF, with the largest gains appearing in Recall@5 and Recall@10. This supports the interpretation that the metadata signal contributes most clearly to coverage-oriented retrieval.

E. Interpretation

Several observations should be stated carefully.

First, LSA provides the strongest top-rank performance in this 10,000-report experiment. It achieves the highest MRR and Hit@1, which means its latent representation places the first relevant candidate slightly earlier than the other methods under the current proxy relevance protocol. One result I did not initially expect was that LSA would outperform the hybrid on both MRR and Hit@1. This suggests that the reduced latent representation helped place the first relevant result earlier, even though LSA is not always guaranteed to improve bug report retrieval.

Second, the proposed hybrid model performs best on Recall@5 and Recall@10. This is the main evidence supporting the lexical-metadata fusion. The hybrid does not dominate the first-rank metrics, but it improves the chance that relevant candidates appear within a short ranked list. In other words, the metadata signal seems to help bring more possible matches into the top 5 or top 10, even if it does not always move the best match to rank 1.

Third, TF–IDF remains a strong and simple lexical baseline. Although it is not the best model on this run, its performance is competitive and it remains easier to interpret than latent or neural representations. This supports the value of sparse lexical retrieval as a baseline in duplicate bug report studies.

Fourth, metadata alone was too weak to work well by itself. Its low MRR and Hit@1 show that component agreement is too coarse to reliably place the best candidate at the top. However, its contribution inside the hybrid suggests that metadata can still help when it is used as a supporting signal rather than as the entire retrieval method.

The Recall@k values are low in absolute terms. This is mainly due to the proxy relevance design and the size and

TABLE III
ABLATION SUMMARY.

Configuration	Observation
TF–IDF only	Strong and interpretable lexical baseline, but not the best on all metrics
Metadata only	Too weak to work well as a standalone ranking signal
LSA only	Best MRR and Hit@1 on the 10,000-report subset
TF–IDF + metadata	Best Recall@5 and Recall@10 among the reported models

TABLE IV
FINAL HYBRID COMPOSITION.

Signal	Weight
TF–IDF similarity	0.85
Metadata similarity	0.15

distribution of product-level relevance groups, not necessarily a direct failure of the retrieval methods. Therefore, the recall values should be interpreted cautiously, especially when comparing them with the higher MRR and Hit@1 values.

Overall, the improvement is useful but not dramatic: the hybrid improves short-list coverage without adding much complexity, but it does not justify a broad claim of universal superiority.

VII. Ablation and Design Implications

The ablation results show that adding signals is not automatically beneficial for every metric. LSA improves top-rank behavior in this subset, while the lexical-metadata hybrid improves short-list coverage. Metadata alone is too coarse, but it becomes useful when combined with lexical similarity. This supports the design choice of treating metadata as a light supporting signal rather than the main retrieval mechanism.

VIII. Efficiency Perspective

Deployment cost is an important motivation for lightweight duplicate bug retrieval. TF–IDF ranking can be implemented with simple indexing and moderate memory. Component matching adds almost no computational overhead. In contrast, transformer-based retrieval usually requires larger memory budgets, heavier inference pipelines, and more engineering effort.

The comparison in Table V is qualitative and is intended to summarize deployment considerations rather than provide exact runtime benchmarking.

IX. Threats to Validity

Several limitations should be stated explicitly.

Dataset scale: The experiments are conducted on a processed 10,000-report subset rather than on the complete public archive. This is large enough for a controlled comparison, but it is still not a full-corpus benchmark.

TABLE V
QUALITATIVE EFFICIENCY COMPARISON.

Model Type	Compute	Memory	Deploy.
TF–IDF	Low	Low	Low
Metadata match	Very low	Very low	Very low
Hybrid (this work)	Low	Low	Low
Transformer retrieval	High	High	High

Hardware-constrained subset selection: The subset size was selected so that repeated local experiments could be completed on the available laptop hardware. In practice, larger pairwise similarity runs became slower and less convenient to repeat. This makes the work reproducible in a constrained setting, but it also restricts how broadly the numerical findings can be generalized.

Proxy relevance grouping: The evaluation uses product-level grouping as a relevance proxy. This avoids direct leakage from the component metadata feature, but it is not equivalent to true duplicate-pair ground truth. The biggest limitation of this study is that product-level grouping is only a proxy for duplicate reports. Reports from the same product are not necessarily duplicates, so the results should be treated as a comparison of retrieval behavior rather than proof of real duplicate detection accuracy.

Feature scope: The metadata signal is intentionally simple and limited to component equality. Other structured fields, such as severity, platform, version, or temporal information, may change the results.

Weight selection: The hybrid weights were chosen using exploratory tuning rather than a separate validation set. This may make the hybrid sensitive to the current subset and should be improved in future work.

Modern semantic baselines: Transformer-based semantic retrieval is not included in this version. Therefore, the paper should be read as a careful lightweight baseline study rather than a complete comparison against modern neural retrieval systems.

X. Conclusion

This paper introduced a lightweight lexical-metadata hybrid for duplicate bug report retrieval. The method combines TF–IDF lexical similarity with a small component-level metadata signal to improve candidate coverage while keeping the model simple, interpretable, and inexpensive to run.

The 10,000-report experiment shows that no single lightweight method dominates all metrics. LSA achieves the strongest top-rank performance, with the best MRR and Hit@1. The proposed hybrid model achieves the best Recall@5 and Recall@10, which suggests that lexical-metadata fusion is most useful as a coverage-oriented retrieval strategy. Metadata alone performs poorly as a standalone model, but it provides useful support when combined with lexical similarity.

The main contribution is therefore practical and carefully bounded. The study does not claim that the hybrid model is

universally superior. Instead, it shows that a simple lexical-metadata fusion can improve short-list retrieval coverage under a controlled proxy relevance protocol. This makes the approach useful as a reproducible baseline, especially when computational simplicity is more important than using a heavy semantic model. Future work should evaluate the approach using verified duplicate-pair labels, larger benchmark settings, temporal validation, statistical significance testing, and stronger semantic baselines.

REFERENCES

[1] P. Runeson, M. Alexandersson, and O. Nyholm, "Detection of duplicate defect reports using natural language processing," in *Proc. 29th Int. Conf. Software Engineering (ICSE)*, 2007, pp. 499–510, doi: 10.1109/ICSE.2007.32.

[2] X. Wang, L. Zhang, T. Xie, J. Anvik, and J. Sun, "An approach to detecting duplicate bug reports using natural language and execution information," in *Proc. 30th Int. Conf. Software Engineering (ICSE)*, 2008, pp. 461–470, doi: 10.1145/1368088.1368151.

[3] N. Jalbert and W. Weimer, "Automated duplicate detection for bug tracking systems," in *Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2008, pp. 52–61.

[4] C. Sun, D. Lo, X. Wang, J. Jiang, and S.-C. Khoo, "A discriminative model approach for accurate duplicate bug report retrieval," in *Proc. 32nd ACM/IEEE Int. Conf. Software Engineering (ICSE)*, 2010, pp. 45–54, doi: 10.1145/1806799.1806811.

[5] C. Sun, D. Lo, S.-C. Khoo, and J. Jiang, "Towards more accurate retrieval of duplicate bug reports," in *Proc. 26th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2011, pp. 253–262, doi: 10.1109/ASE.2011.6100061.

[6] A. Lamkanfi, J. Pérez, and S. Demeyer, "The Eclipse and Mozilla defect tracking dataset: A genuine dataset for mining bug information," in *Proc. 10th IEEE Working Conf. Mining Software Repositories (MSR)*, 2013, pp. 203–206, doi: 10.1109/MSR.2013.6624028.

[7] N. Bettenburg, R. Premraj, T. Zimmermann, and S. Kim, "Extracting structural information from bug reports," in *Proc. Int. Working Conf. Mining Software Repositories (MSR)*, 2008, pp. 27–30, doi: 10.1145/1370750.1370757.

[8] A. J. Ko, B. A. Myers, M. J. Coblenz, and H. H. Aung, "An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks," *IEEE Trans. Softw. Eng.*, vol. 32, no. 12, pp. 971–987, 2006, doi: 10.1109/TSE.2006.116.

[9] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.

[10] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988, doi: 10.1016/0306-4573(88)90021-0.

[11] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009, doi: 10.1561/1500000019.

[12] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer, and R. Harshman, "Indexing by latent semantic analysis," *J. Amer. Soc. Inf. Sci.*, vol. 41, no. 6, pp. 391–407, 1990, doi: 10.1002/(SICI)1097-4571(199009)41:6<391::AID-ASI1>3.0.CO;2-9.

[13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186, doi: 10.18653/v1/N19-1423.

[14] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992, doi: 10.18653/v1/D19-1410.